ZERO SHOT COMPUTER VISION

Kourosh GERAYELI

CEDRE 2026

INTRO TO TRANSFER LEARNING

You transfer what you learned from one instrument to another.
We can use this concept in neural networks as well.

Suppose you learned how to drive a **car**.

You spent months learning:

- How steering works
- How acceleration feels
- How braking distance changes with speed
- How to read mirrors
- How to anticipate traffic

Now one day, someone gives you a **truck**.

At first, it feels different:

- It's bigger
- It turns wider
- It brakes slower
- It sits higher

But here's the important part:

You do **not** need to relearn:

- What a steering wheel does
- What a brake pedal does
- What traffic signs mean
- How lanes work

You already understand the *fundamentals of driving*.

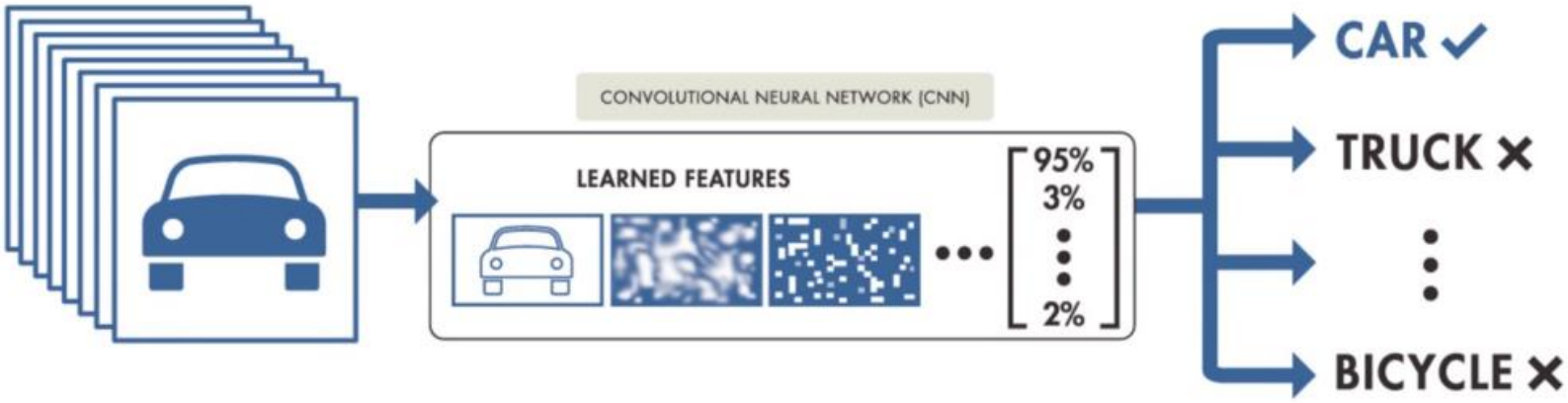
You only need to adjust:

- Turning radius
- Braking timing
- Awareness of vehicle size

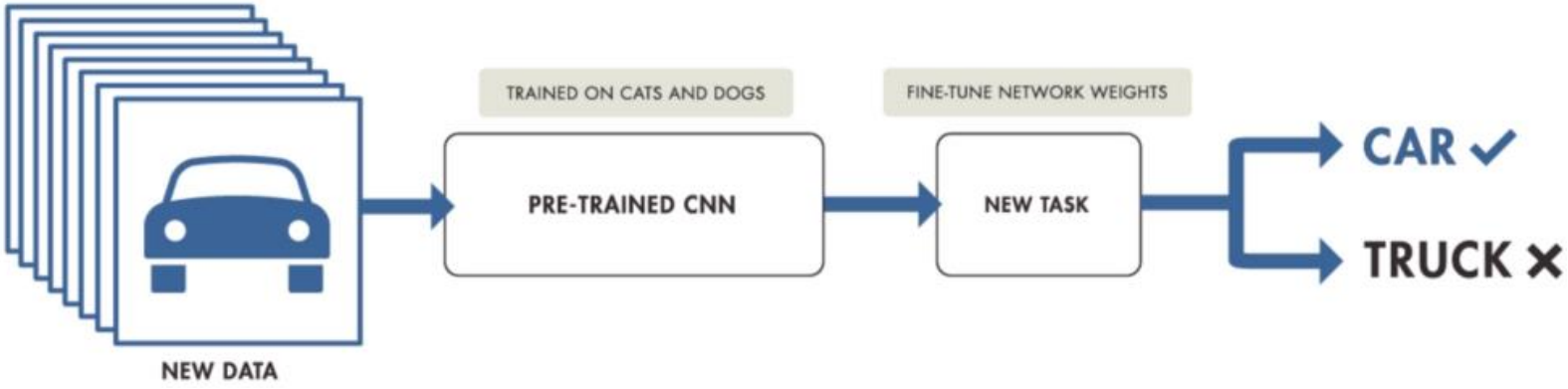
You transfer your driving knowledge from car → truck.

You don't start from zero.

TRAINING FROM SCRATCH

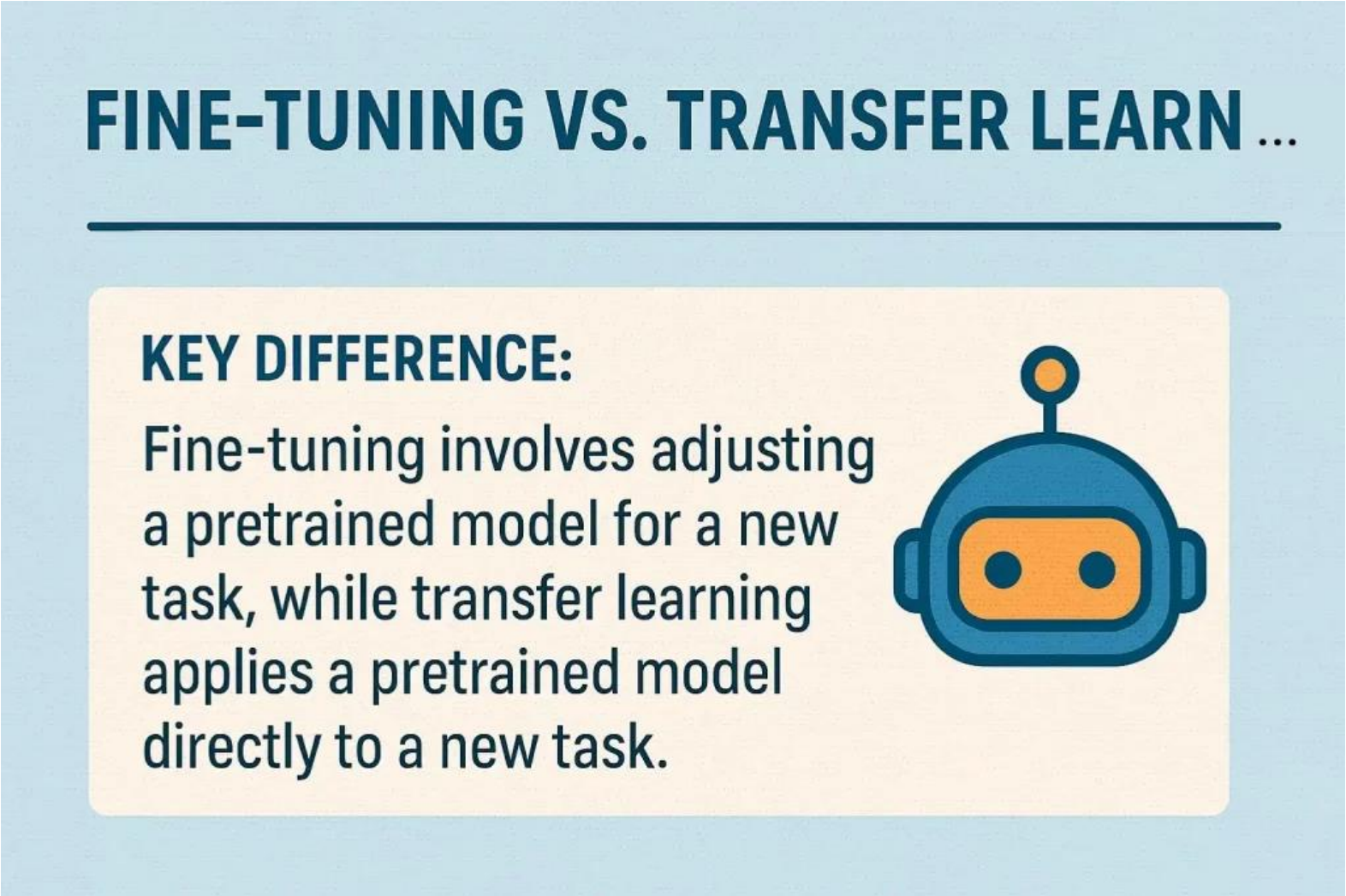


TRANSFER LEARNING



Transfer learning requires that the previous knowledge is “useful” for the other task. Thus, the features we are trying to explore need to be general enough for the new application. If we go back to our car driving example, driving the airplane instead of the car is not as helpful in learning how to drive the truck. The main skill that gives the car driver its head start is the intuitive understanding of where pedals are for example. Yet, the car player is not starting from zero. He is familiar with things like steering wheel, rhythm, and timing. These general skills give them an edge over someone who never driven a car at all. The act of playing an instrument gives all players a general set of skills that are useful across instruments. This generalization across domains (in our example, musical instruments) is what makes the model learn much faster as opposed to training from zero.

TRANSFER LEARNING AND FINE TUNING



Let's make a distinction between the concepts we are talking about.

Transfer Learning (Feature Extraction)

Idea:

Use a **pretrained model** (trained on a large dataset like ImageNet) as a **feature extractor**, and **only train a new final layer** for your task.

What happens

•Load a pretrained model (e.g., ResNet or Vision Transformer).

•**Freeze** all the pretrained layers.

•Replace the **last classification layer**.

•Train **only the new layer**.

Simple Example

Suppose a model was trained to classify **1000 objects** on ImageNet.

Now you want to classify **cats vs dogs**.

Steps:

1.Load pretrained model

2.Freeze its layers

3.Replace the last layer with **2 outputs**

4.Train only this layer

Fine-Tuning

Idea:

Start with a pretrained model but **continue training some or all layers** on your dataset.

What happens

•Load pretrained model

•Replace the final layer

•**Unfreeze some layers**

•Train them on your dataset

Imagine a **chef trained in Italian cuisine**.

Transfer Learning

You hire the chef and **only change the menu**.

The chef keeps all cooking habits.

Fine-Tuning

You hire the chef **and retrain them slightly** to cook **French food**.

Their cooking style adapts.

ZERO SHOT LEARNING

Zero-shot learning is a setup in which the model is presented with images belonging to **only** classes that it was not exposed to during training, at test time

HOW DOES ZERO-SHOT LEARNING WORK IN COMPUTER VISION?

how can we humans do it?
How can we recognize objects that we have not seen before?

Difference between zero shot learning in NLP and computer vision. In NLP since they have been trained on huge amount of dataset they can predict the next word like in your phone! But in computer vision it is whole another story.

We need some **other information** about that object. We can easily identify a tiger, even if we haven't seen one before, given that we know what a lion looks like. A tiger is a lion with stripes and minus the mane. A zebra is a black-and-white striped version of a horse. Deadpool is a red-black version of Spiderman.

OTHER INFORMATION

It provides a semantic bridge between the visual features and the unseen classes. By incorporating this multi-modal information (Text & Image), zero-shot computer vision models can learn to recognize objects even if they have never seen them before visually. In other words, semantic information embeds **both** seen and unseen classes in high-dimensional vectors, and it comes in many different forms:

1. **Attribute vectors**: Think of attributes as a tabular representation of the different features of the object.
2. **Textual Descriptions**: A text describing the object contained in the image similar to image captions.
3. **Class label vectors**: Those are the embeddings of the class labels themselves.

ZERO SHOT VS TRANSFER LEARNING

Zero-shot learning (ZSL) falls within the broader category of transfer learning, specifically under the umbrella of **heterogeneous transfer learning**. This means that ZSL relies on transferring knowledge gained from a **source domain** (seen classes) to a **distinct target domain** (unseen classes).

METHODS OF ZERO-SHOT COMPUTER VISION

- **Embedding-based methods**: The model learns a common embedding space where both images and their semantic features/representations are projected. New unseen classes are then predicted using a similarity measure in that space. For example, CLIP.
- **Generative-based methods**: These methods utilize generative models to create synthetic images or visual features for the unseen classes based on the samples of seen classes and semantic representations of both classes. This allows for training models on these unseen classes even without real data.

WHY IS ZERO-SHOT LEARNING USEFUL?

- Zero-Shot Learning is useful because it can help to overcome the challenges and costs associated with data labeling, an time-consuming and often expensive process. It is especially difficult to obtain annotations from specialized experts in certain fields, such as biomedical data, which requires the expertise of trained medical professionals.
- Additionally, there may not be enough training data available for a class (i.e. in the case of trying to label rare defects in products), or the data may be imbalanced. These two scenarios make it difficult for a model to accurately reflect real-world scenarios. Furthermore, unsupervised learning methods may also be insufficient in some tasks such as classifying different sub-categories of the same object (for example, breeds of a dog).
- ZSL can help to alleviate these problems by allowing a model to perform classification on novel classes (unseen classes) using the knowledge it has already learned during training.

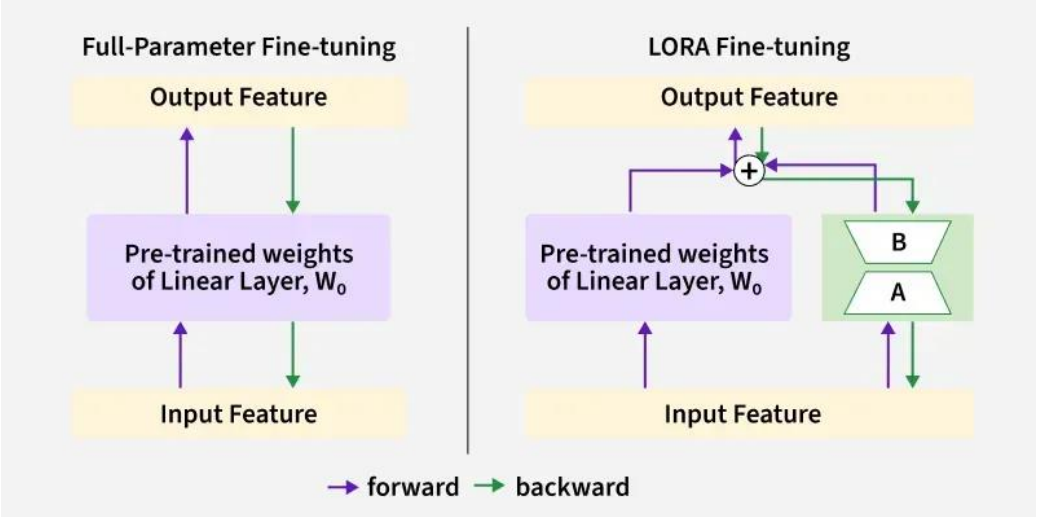
LORA

LORA : A way to fine-tune a big neural network without changing all its weights.

Low-rank adaptation: is a method for rapidly adapting machine learning models to new use cases without retraining them. It enables developers to customize models for specific contexts. LoRA works by appending a lightweight addition, called a low-rank matrix, to the original model.

Instead of directly modifying the model's full weight matrices, LoRA introduces a low-rank decomposition, significantly reducing the number of trainable parameters.

LoRA adds low-rank matrices to the frozen original machine learning model. These matrices contain new weights to apply to the model when generating results. This process alters the outputs that the model produces with minimal computing power and training time.



Large Language Models (LLMs) like ChatGPT, Gemini, and Llama have revolutionized text processing thanks to their flexibility and **zero-shot learning** capabilities. While some specialized tasks may benefit from fine-tuning — **with LoRA being the most efficient approach** — the vast majority of applications can leverage LLMs **out-of-the-box** through effective prompting.
These models have also streamlined AI prototyping. Just a few years ago, solving a basic task like document classification required multiple steps: extracting text descriptors like bag-of-words or embeddings, and training a classifier such as an SVM, Random Forest, or a Deep Neural Network. Today, an LLM combined with **good prompt engineering** can handle the entire task with minimal effort. This paradigm shift extends to summarization, translation, entity extraction, and many other NLP tasks — **all achievable through LLMs and effective prompts.**
Vision-language models (VLMs) are bringing a **similar transformation** to computer vision. By integrating visual and textual data, these models excel in tasks like **image captioning, visual question answering, and text-to-image retrieval**, significantly improving contextual understanding and performance. As these models become increasingly relevant, it is **crucial** for computer vision experts to understand how to **modify and adapt** them for specialized use cases.
The most efficient way to **adapt a generative model** — whether for text, image, or multimodal tasks — is through **LoRA**, as it enables targeted fine-tuning with minimal computational overhead. Let's explore how to make the most of this technique.

Large and complex machine learning models, such as those used for large language models (LLMs) like ChatGPT, take a long time and a lot of resources to set up. They may have trillions of parameters that are set to specific values. Once this process is complete, the model may be powerful and accurate in general, but it is not necessarily fine-tuned to carry out specific tasks.
Getting a model to work in specific contexts can require a great deal of retraining, changing all its parameters. With the amount of parameters in such models, this retraining is expensive and time-consuming. LoRA provides a quick way to adapt the model without retraining it.

Core Idea

Instead of training :

$$W$$

We train :

$$W + \Delta W$$

But instead of learning full ΔW, we force it to be:

$$\Delta W = A \times B$$

Where A and B are small matrices (Low rank).

A linear layer looks like:

$$y = xW$$

If:

Input size = 768
Output size = 768
Then:

$$W \in R^{768*768}$$

➔ 589,824 paramteres for ONE matrix.
➔ Very biG!!!

LoRA says:

Instead of learning a full correction matrix

$$\Delta W \in R^{768*768}$$

We force it to be:

$$\Delta W = AB$$

Where:

$$A \in R^{768*r}$$
$$B \in R^{r*768}$$

And r is small like :
r = 4,8

➔ So instead of 589k parameters, if r = 8 → (768 × 8) = 6144 parameters for A and for B ➔ 12,288 parameters in total ➔ ~ 2% of the original matrix

But What Are A and B Conceptually?

They are:

👉 Just normal trainable weight matrices

👉 Randomly initialized

👉 Learned using backpropagation

They are NOT computed from W.

They are NOT derived from SVD.

They are learned during training.

Think of W as operating in 768-dimensional space.

LoRA says:

Instead of allowing movement in all 768 directions,

we restrict updates to only r directions.

A decides:

👉 which directions are allowed.

B decides:

👉 how strong each direction affects outputs.

So:

•A = basis directions

•B = how we combine them

A and B are:

- Small trainable matrices
- Randomly initialized
- Learned by backprop
- Designed so that $\Delta W = AB$ is low rank
- Used to gently adjust frozen pretrained weights

They are NOT:

•Precomputed

•Extracted from W

•Special mathematically derived matrices

They are just smartly structured trainable parameters

Notebook loRa